Universidad Nacional de San Antonio Abad del Cusco

Departamento Académico de Ingeniería Informática y de Sistemas

Desarrollo de Software II

PRÁCTICA 06

Diseño Responsivo y Accesible en Flutter

Todo lo que necesitas para crear apps que se adaptan a cualquier pantalla y son usables por todas las personas.

1. Diseño Responsivo

Flutter ofrece un conjunto de widgets que permiten adaptar la interfaz de usuario a cualquier tamaño de pantalla, desde teléfonos pequeños hasta monitores de escritorio, sin necesidad de escribir código específico por plataforma.

LAYOUTBUILDER

Adaptar layout según espacio disponible

El widget más fundamental para diseño responsivo. Recibe las restricciones reales del widget padre, no del dispositivo completo. Úsalo cuando necesitas tomar decisiones de layout basadas en el espacio disponible del contenedor.

```
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth > 600) {
      // Layout para tablet/desktop
      return Row(children: [...]);
    }
    // Layout para móvil
    return Column(children: [...]);
  },
)
```

CUÁNDO USARLO

Úsalo cuando necesitas tomar decisiones de layout basadas en el espacio disponible del contenedor, no del dispositivo entero. Esto es especialmente útil para widgets reutilizables.

MEDIAQUERY

Información del dispositivo y pantalla

Accede al tamaño de pantalla, orientación, preferencias del usuario como el modo oscuro, y mucho más. En Flutter 3.10+, usa los métodos específicos para optimizar los rebuilds.

```
final size = MediaQuery.of(context).size;
final isLandscape = MediaQuery.of(context)
    .orientation == Orientation.landscape;
final textScale = MediaQuery.of(context).textScaleFactor;

// En Flutter 3.10+ (más eficiente, menos rebuilds)
final width = MediaQuery.sizeOf(context).width;
```

FRACTIONALLYSIZEDBOX

Tamaños proporcionales

Define dimensiones como fracción del espacio disponible. Ideal para evitar anchos fijos que rompen el layout en pantallas pequeñas o muy grandes.

```
FractionallySizedBox(
  widthFactor: 0.8,  // 80% del padre
  child: ElevatedButton(
    onPressed: () {},
    child: Text('Continuar'),
  ),
)
```

FLEXIBLE Y EXPANDED

Distribución de espacio en Row/Column

Expanded ocupa todo el espacio disponible. Flexible ocupa el que necesita, hasta un máximo. Ambos aceptan un parámetro flex para controlar proporciones.

```
Row(children: [
  Expanded(
    flex: 2,
    child: TextField(),  // 2/3 del ancho
  ),
  SizedBox(width: 8),
  Expanded(
    flex: 1,
    child: ElevatedButton(  // 1/3 del ancho
      onPressed: () {},
      child: Text('Buscar'),
    ),
  ),
```

```
])
```

2. Breakpoints y Adaptive Design

Organizar los breakpoints en una clase centralizada hace que el código sea más mantenible y consistente en toda la app.

CLASE DE BREAKPOINTS

Define tus breakpoints en un solo lugar

```
class Breakpoints {
  static const double mobile  = 480;
  static const double tablet  = 768;
  static const double desktop = 1024;
  static const double wide    = 1440;

  static bool isMobile(BuildContext ctx) =>
      MediaQuery.sizeOf(ctx).width < mobile;

  static bool isTablet(BuildContext ctx) {
    final w = MediaQuery.sizeOf(ctx).width;
    return w >= mobile && w < desktop;
  }

  static bool isDesktop(BuildContext ctx) =>
      MediaQuery.sizeOf(ctx).width >= desktop;
}
```

ADAPTIVEGRID

Grid adaptable con columnas dinámicas

Cambia el número de columnas automáticamente según el ancho disponible, sin necesidad de verificar el breakpoint manualmente.

```
GridView.builder(
  gridDelegate: SliverGridDelegateWithMaxCrossAxisExtent(
    maxCrossAxisExtent: 200,  // máx. ancho por celda
    childAspectRatio: 1.0,
    crossAxisSpacing: 16,
    mainAxisSpacing: 16,
  ),
  itemCount: items.length,
  itemBuilder: (ctx, i) => ItemCard(items[i]),
)

// Alternativa con columnas explícitas
LayoutBuilder(builder: (ctx, constraints) {
```

```
  final cols = (constraints.maxWidth / 200).floor()
      .clamp(1, 4);
  return GridView.count(crossAxisCount: cols, ...);
})
```

NAVIGATIONRAIL

Navegación adaptable según breakpoint

Cambia entre BottomNavigationBar para móvil y NavigationRail para pantallas más anchas, ofreciendo la experiencia óptima en cada dispositivo.

```
Scaffold(
  body: Row(children: [
    if (Breakpoints.isDesktop(context))
      NavigationRail(
        destinations: [...],
        selectedIndex: _selectedIndex,
        onDestinationSelected: (i) =>
            setState(() => _selectedIndex = i),
        labelType: NavigationRailLabelType.all,
      ),
    Expanded(child: _pages[_selectedIndex]),
  ]),
  bottomNavigationBar: Breakpoints.isMobile(context)
    ? BottomNavigationBar(...)
    : null,
)
```

EVITAR

No uses tamaños fijos en píxeles para anchos de contenedores principales. Siempre usa Expanded, FractionallySizedBox, o restricciones relativas.

3. Accesibilidad

Flutter tiene soporte nativo para lectores de pantalla, zoom de texto y navegación por teclado. Diseñar con accesibilidad en mente desde el inicio es siempre más eficiente que corregirla después.

4.5:1	44px
Contraste mínimo WCAG AA	Tamaño mínimo de touch target

TEXTO ESCALABLE

Respetar la escala de texto del usuario

Nunca bloquees el escalado de texto. Los usuarios con discapacidad visual dependen de esta función del sistema operativo.

```
// Correcto - permite escalar
Text(
  'Hola mundo',
  style: TextStyle(fontSize: 16),
)

// Evitar - bloquea escalado
Text(
  'Hola mundo',
  textScaleFactor: 1.0,  // no hagas esto
)

// Si necesitas limitar, establece un máximo razonable
MediaQuery(
  data: MediaQuery.of(context).copyWith(
    textScaleFactor: MediaQuery.of(context)
        .textScaleFactor.clamp(1.0, 1.4),
  ),
  child: child,
)
```

FOCUS Y TECLADO

Navegación por teclado y control de foco

Asegúrate de que toda la UI sea navegable sin pantalla táctil. Esto es especialmente importante en tablets con teclado y en apps de escritorio.

```
Focus(
  onFocusChange: (hasFocus) {
    // Feedback visual explícito al recibir foco
  },
  child: InkWell(
    onTap: () {},
    focusColor: Colors.blue.withOpacity(0.1),
    child: Container(...),
  ),
)

// Orden de foco personalizado
FocusTraversalOrder(
  order: NumericFocusOrder(1.0),
  child: TextField(),
)
```

TOUCH TARGETS

Áreas de toque suficientemente grandes

El icono puede ser pequeño, pero el área de toque debe ser de al menos 44x44 píxeles lógicos para facilitar la interacción.

```
// Usar SizedBox para agrandar el área de toque
SizedBox(
  width: 48,
  height: 48,
  child: IconButton(
    onPressed: () {},
    icon: Icon(Icons.close, size: 24),
    // El icono es 24px, pero el área de toque es 48px
  ),
)

// O con GestureDetector y padding
Padding(
  padding: EdgeInsets.all(12),
  child: GestureDetector(onTap: () {}, child: ...),
)
```

4. Semántica para Lectores de Pantalla

Flutter usa el árbol semántico para comunicar la UI a TalkBack (Android) y VoiceOver (iOS). Sin semántica correcta, los usuarios de lectores de pantalla no pueden usar la app.

SEMANTICS WIDGET

Añadir contexto a widgets sin texto

Cualquier widget interactivo que no tenga texto visible necesita una etiqueta semántica. Los iconos, imágenes y gestos personalizados son los candidatos más comunes.

```
Semantics(
  label: 'Foto de perfil de María García',
  hint: 'Doble toque para cambiar la foto',
  button: true,
  child: GestureDetector(
    onTap: _changePhoto,
    child: CircleAvatar(
      backgroundImage: NetworkImage(profileUrl),
    ),
  ),
)

// Para iconos decorativos, excluirlos del árbol
ExcludeSemantics(
  child: Icon(Icons.star, color: Colors.amber),
)
```

MERGESEMANTICS

Agrupar widgets relacionados

Combina el árbol semántico de varios widgets en uno solo para evitar lecturas fragmentadas que confunden al usuario.

```
MergeSemantics(
  child: Row(children: [
    Checkbox(value: checked, onChanged: _toggle),
    Text('Recordar contraseña'),
  ]),
)
// El lector lee: "Recordar contraseña, casilla de verificación"
// en lugar de leer el checkbox y el texto por separado
```

ANUNCIOS DINÁMICOS

Notificar cambios de estado

Cuando el estado cambia sin navegación —por ejemplo, al añadir un producto al carrito— anuncia el cambio al lector de pantalla.

```
// Anunciar mensaje sin enfocar ningún widget
SemanticsService.announce(
  'Producto añadido al carrito',
  TextDirection.ltr,
);

// O con Semantics y liveRegion
Semantics(
  liveRegion: true,
  child: _isLoading
      ? Text('Cargando...')
      : Text('Listo'),
)
```

HERRAMIENTA DE PRUEBA

Usa el análisis de accesibilidad en Flutter DevTools para visualizar el árbol semántico sin necesitar un lector de pantalla real. También puedes activar la superposición semántica con SemanticsDebugger.

5. Checklist de Lanzamiento

Revisa estos puntos antes de publicar tu app. Tener todos marcados garantiza una experiencia sólida para todos los usuarios.

Diseño responsivo

- ▢ No hay anchos fijos en los widgets principales del layout
- ▢ La UI funciona correctamente en modo horizontal y vertical
- ▢ El contenido es legible y usable en pantallas de 320px a 1440px
- ▢ Las imágenes no se desbordan ni distorsionan en pantallas pequeñas
- ▢ Los grids y listas adaptan sus columnas al espacio disponible
- ▢ La navegación principal cambia según el breakpoint

Texto y tipografía

- ▢ El texto es legible con textScaleFactor de 1.5x (mínimo)
- ▢ No se usa textScaleFactor fijo que bloquee el escalado del usuario
- ▢ Los textos tienen contraste de al menos 4.5:1 sobre su fondo
- ▢ El tamaño mínimo de fuente es 12px en su escala base

Accesibilidad

- ▢ Toda la funcionalidad es navegable con teclado externo
- ▢ Todos los iconos interactivos tienen etiqueta semántica
- ▢ Los touch targets tienen al menos 44x44 píxeles lógicos
- ▢ Las imágenes decorativas están excluidas del árbol semántico
- ▢ Los cambios de estado se anuncian con SemanticsService o liveRegion
- ▢ La app fue probada con TalkBack (Android) y VoiceOver (iOS)
- ▢ No se usa solo el color para transmitir información

Pruebas

- ▢ Tests con flutter_test verifican la semántica de widgets críticos
- ▢ La app fue probada en un dispositivo físico pequeño (5")
- ▢ La app fue probada en tablet o emulador grande
- ▢ Se ejecutó el análisis de accesibilidad en Flutter DevTools